

Convolutional Neural Networks and Computer Vision

CNN

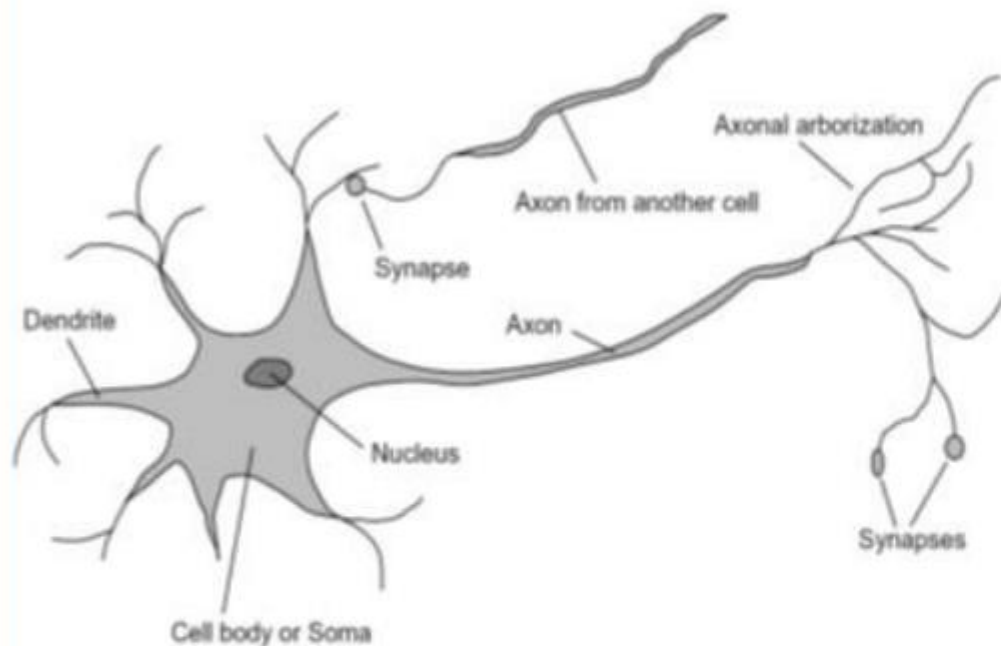
Convolutional neural networks are deep artificial neural networks that are used primarily to classify images, cluster them by similarity, and perform object recognition within scenes.

By use of CNNs it can identify faces, individuals, any signs, tumors and many other aspects of visual data..

CNN

Convolutional Neural Networks are complex feed forward neural networks. CNNs are used for image classification and recognition because of its high accuracy. It was proposed by computer scientist Yann LeCun in the late 90s, when he was inspired from the human visual perception of recognizing things.

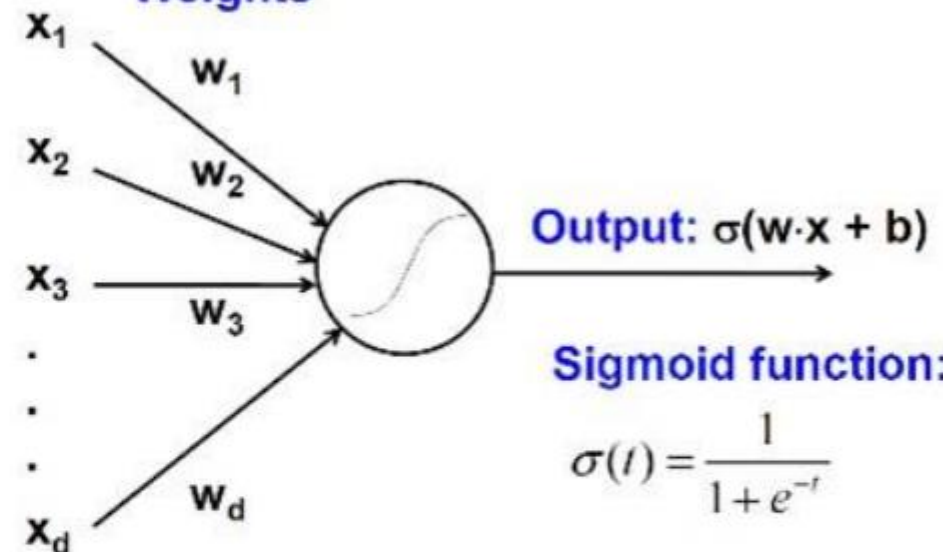
Biological neuron and Perceptrons



A biological neuron

Input

Weights



An artificial neuron (Perceptron)
- a linear classifier



Neural Networks: Layers and Functionality

In a regular Neural Network there are three types of layers:

Input Layers: It's the layer in which we give input to our model. The number of neurons in this layer is equal to the total number of features in our data (number of pixels in the case of an image).

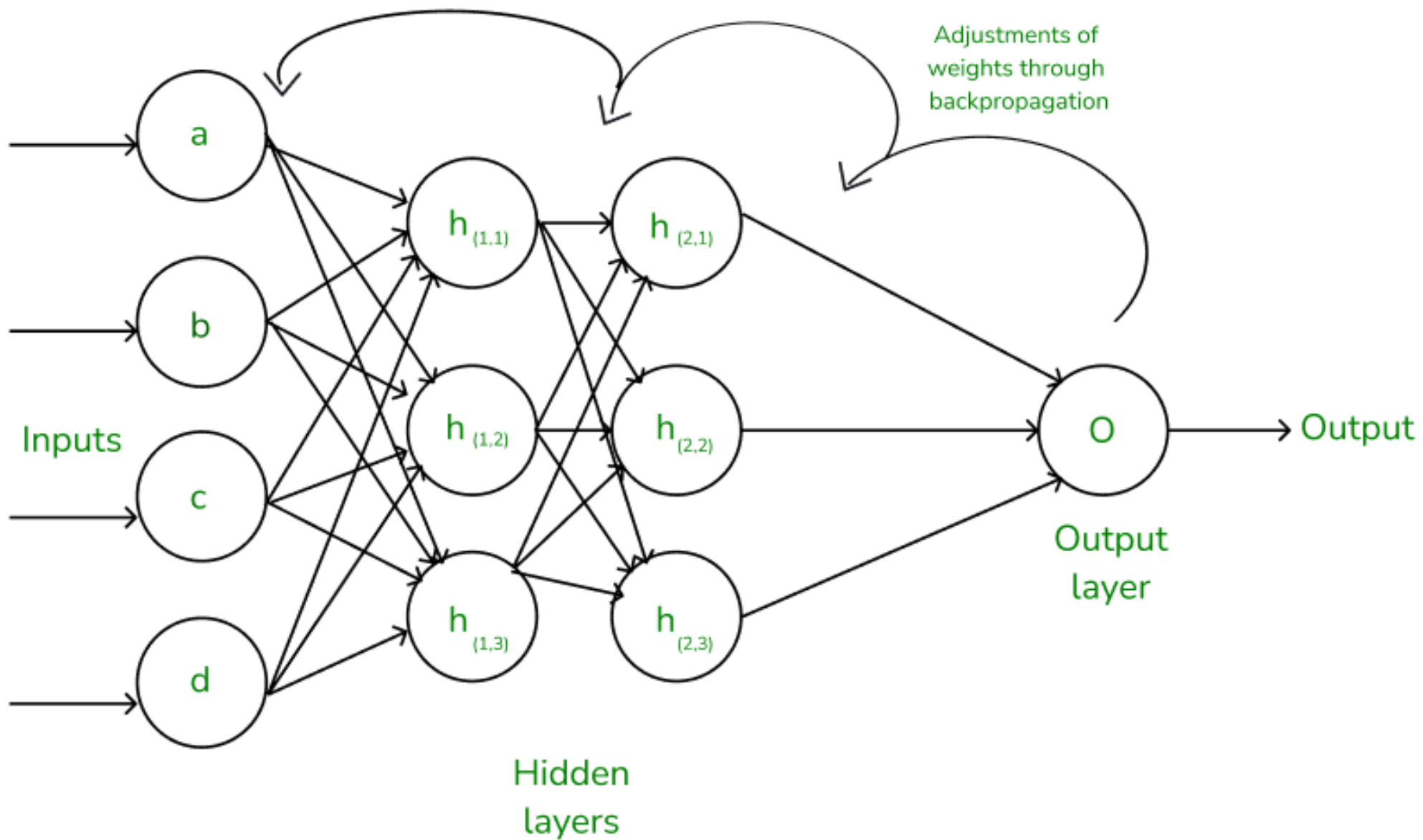
Hidden Layer: The input from the Input layer is then fed into the hidden layer. There can be many hidden layers depending on our model and data size. Each hidden layer can have different numbers of neurons which are generally greater than the number of features. The output from each layer is computed by matrix multiplication of the output of the previous layer with learnable weights of that layer and then by the addition of learnable biases followed by activation function which makes the network nonlinear.

Neural Networks: Layers and Functionality

Output Layer: The output from the hidden layer is then fed into a logistic function like sigmoid or softmax which converts the output of each class into the probability score of each class.

The data is fed into the model and output from each layer is obtained from the above step is called feedforward, we then **calculate the error using an error function**, some common error functions are cross-entropy, square loss error, etc. The error function measures how well the network is performing. After that, we backpropagate into the model by calculating the derivatives. This step is called Backpropagation which basically is used to minimize the loss.

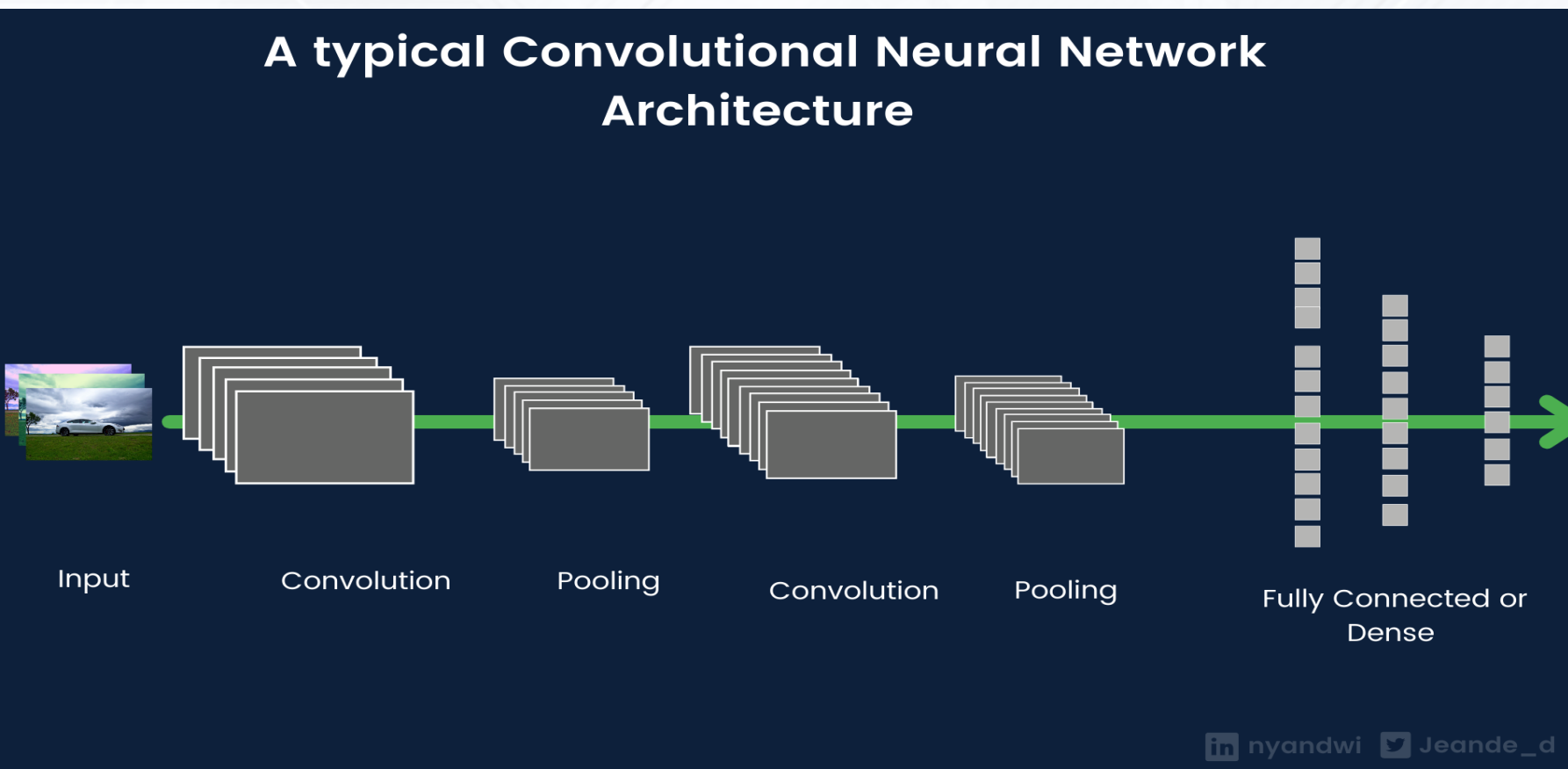
Backpropagation to Minimize Loss



A Typical Architecture of Convolutional Neural Networks

CNN is made of 3 main layers, in sequential order:

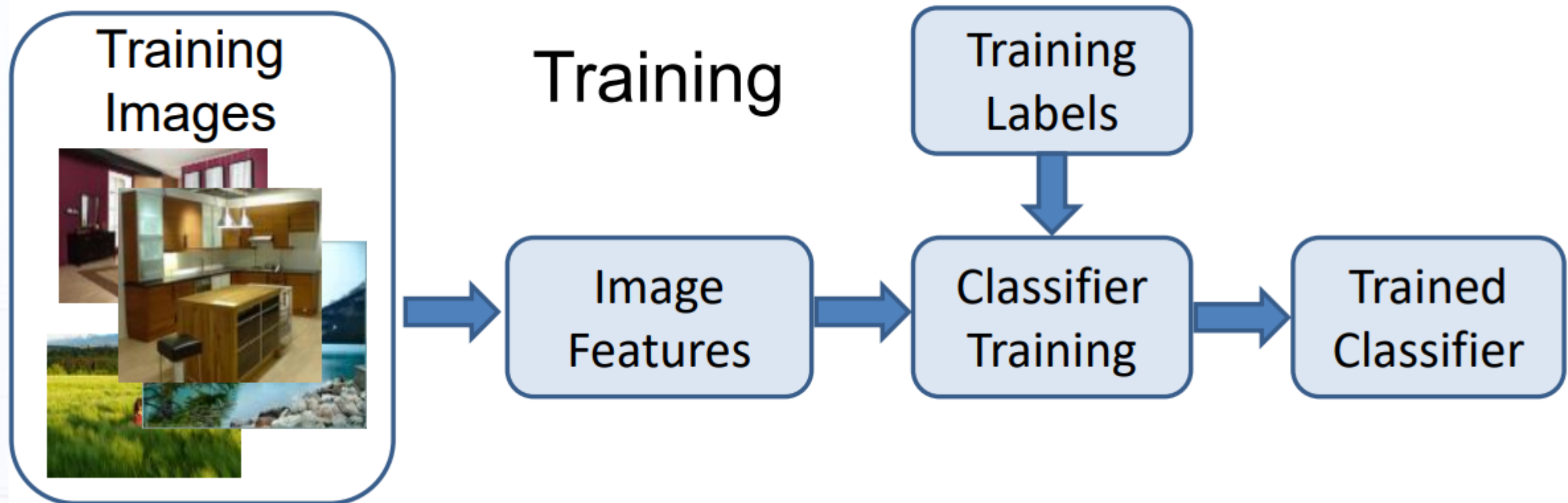
1. **Convolutional layer**
2. **Pooling layer**
3. **Fully connected layer**



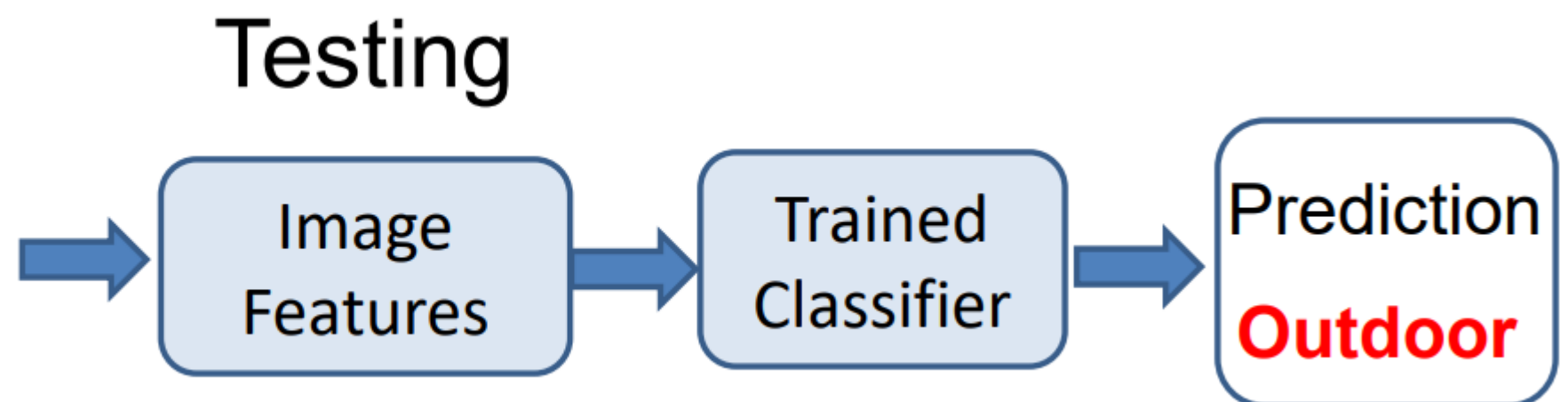
Convolutional layers

- The convolution layers are the backbone of the whole CNN. They are used to extract the features in the images using filters.
- These filters learn both **low-level features** such as lines, edges and **high-level features** such as the face, ear, nose, etc...High-level features are what later become useful during image recognition.

Image Categorization: Training phase

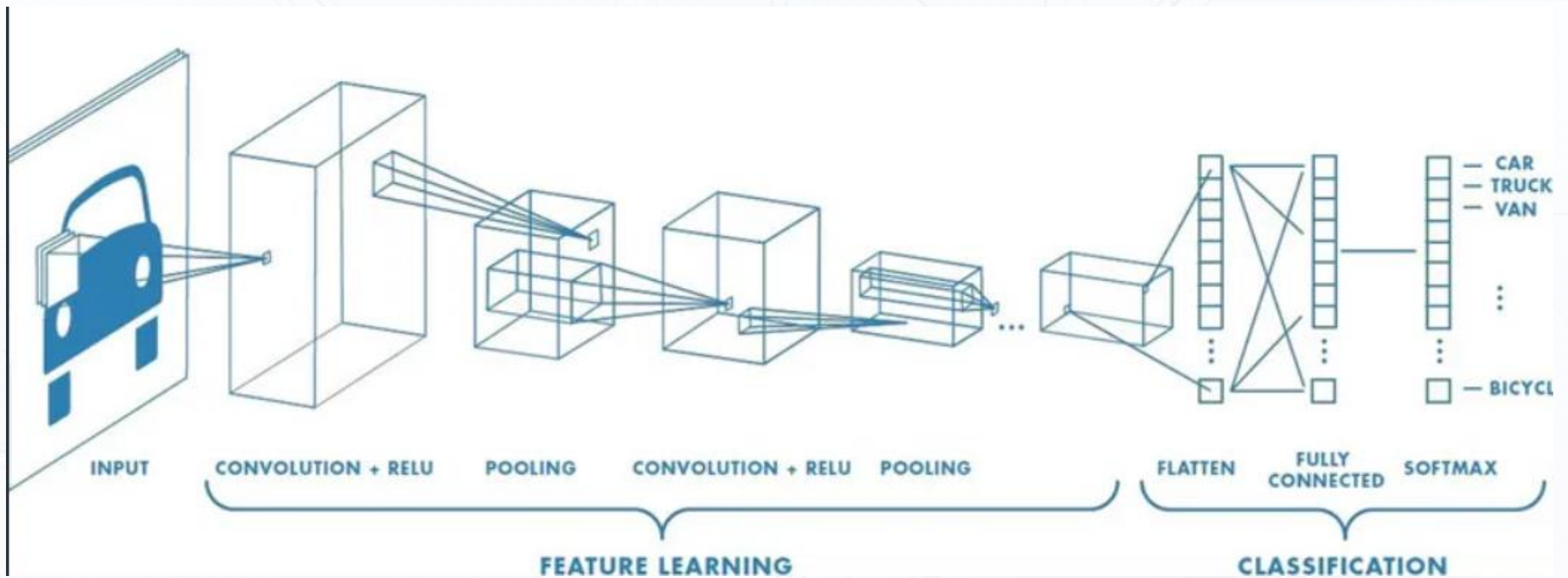


Test Image



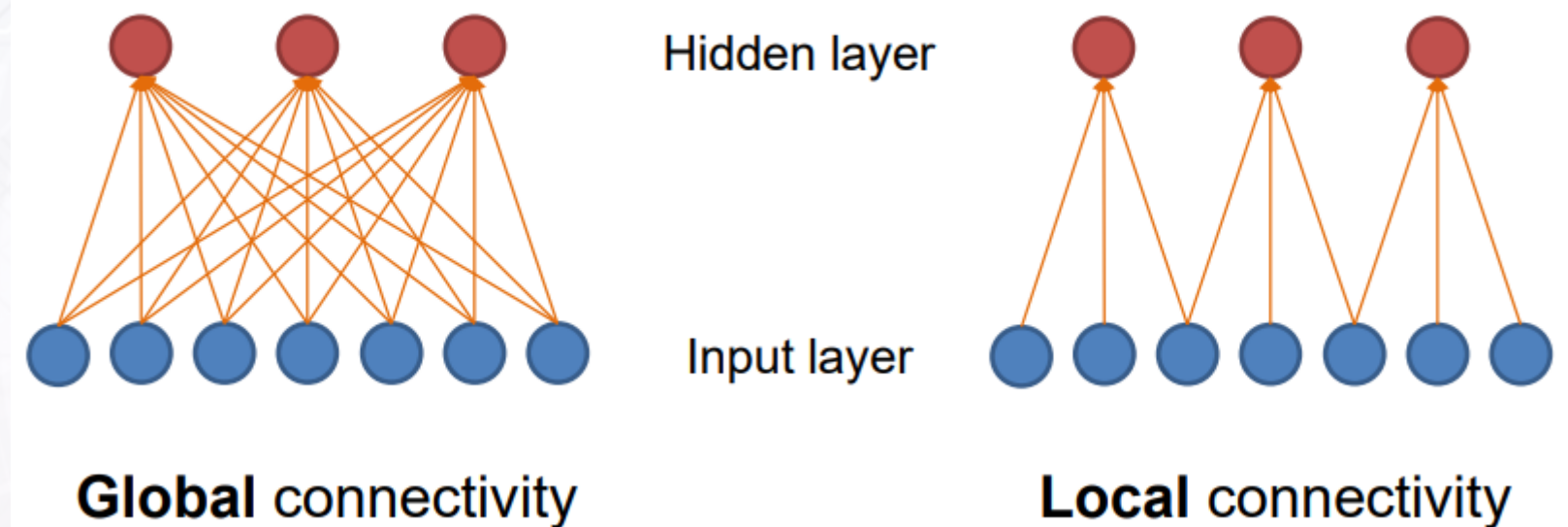
CNN

Convolutional

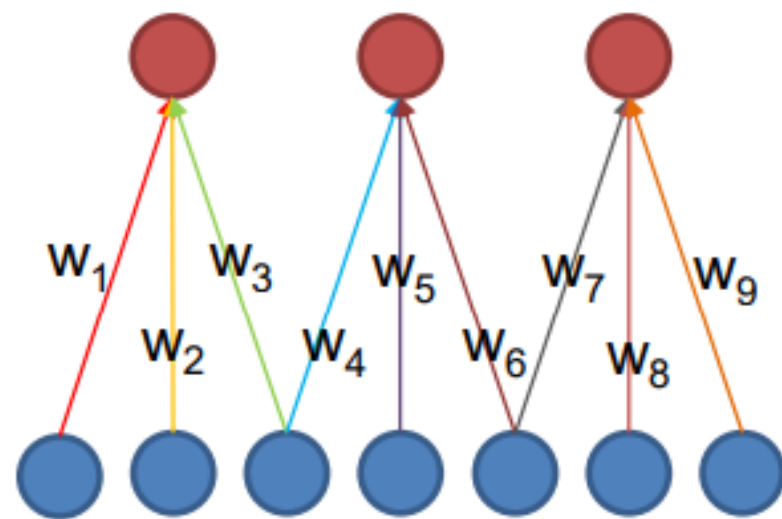


CNN: Local Connectivity

- # input units (neurons): 7
- # hidden units: 3
- Number of parameters
 - Global connectivity: $3 \times 7 = 21$
 - Local connectivity: $3 \times 3 = 9$



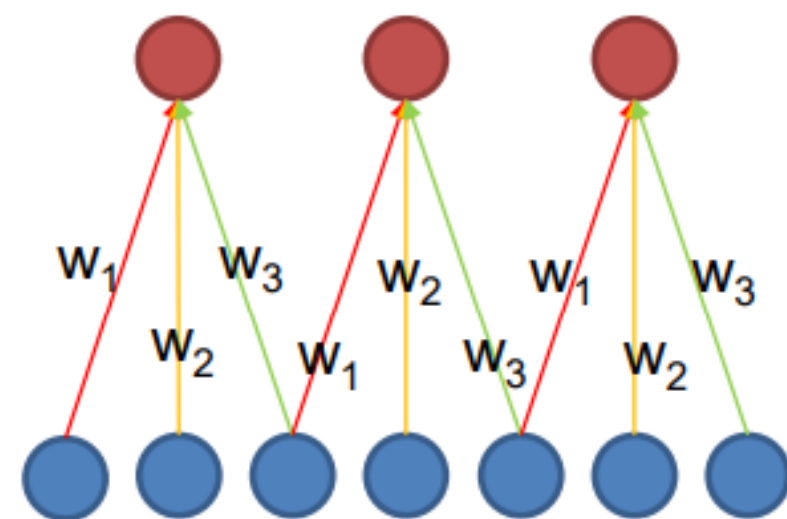
CNN: Weight Sharing



Without weight sharing

Hidden layer

Input layer



With weight sharing

CNN: Weight Sharing

In traditional neural networks, each neuron in one layer is connected to every neuron in the next layer, and each of these connections has its own weight. This results in a massive number of parameters, especially for large input sizes, making the network prone to overfitting and computationally expensive.

CNNs address this issue through the weight-sharing mechanism. In this approach, the same weights are used across different parts of the input, significantly reducing the number of parameters.

CNN: Weight Sharing

Advantages

Reduced Complexity: By sharing weights, CNNs drastically reduce the number of parameters, making the network less complex and easier to train.

Translation Invariance: Weight-sharing helps in detecting features regardless of their position in the input space, contributing to the translation invariance property of CNNs.

Efficiency: With fewer parameters, CNNs are more computationally efficient and require less memory.

CNN: Weight Sharing

Disadvantages

Limited Perception: Due to their local receptive field, individual neurons in a CNN might have a limited understanding of the overall context.

Spatial Invariance Limitation: While good at handling translation, CNNs are less effective with other transformations like rotation and scaling without additional augmentation.

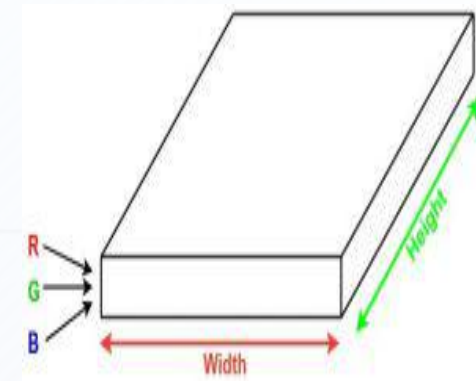
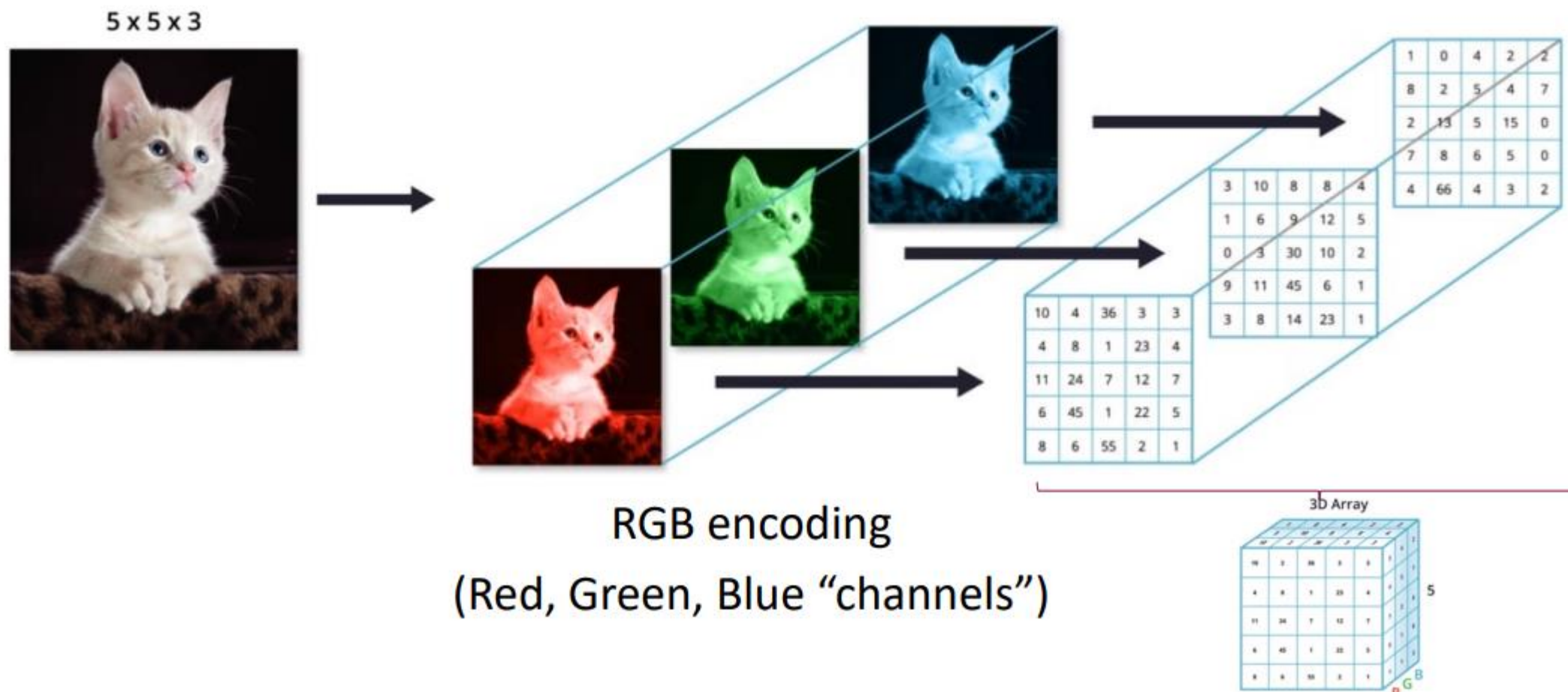
Conclusion

The weight-sharing property of CNNs has enabled advancements in numerous applications, including image and video recognition, medical image analysis, and autonomous driving.

How Convolutional Layers Works?

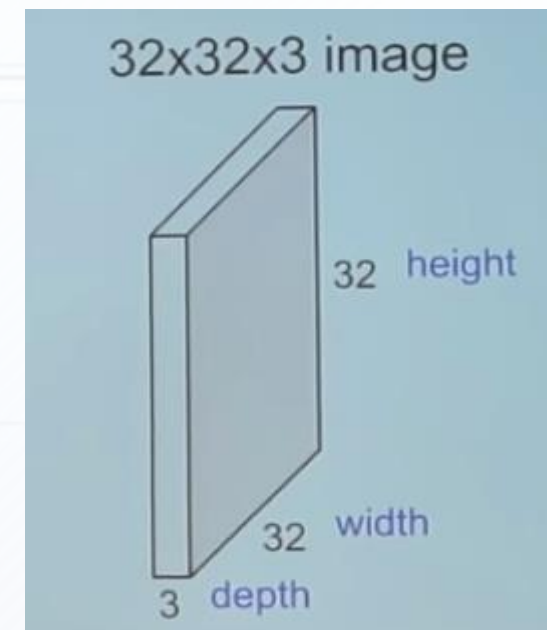
Imagine you have an image. It can be represented as a cuboid having its length, width (dimension of the image), and height (i.e **the channel as images generally have red, green, and blue channels**).

Color Images Are 3D Arrays with 2 Spatial Dimensions



How Convolutional Layers Works?

Unlike neural networks, where the input is a vector, here the input is a multi-channelled image (3 channelled in this case).

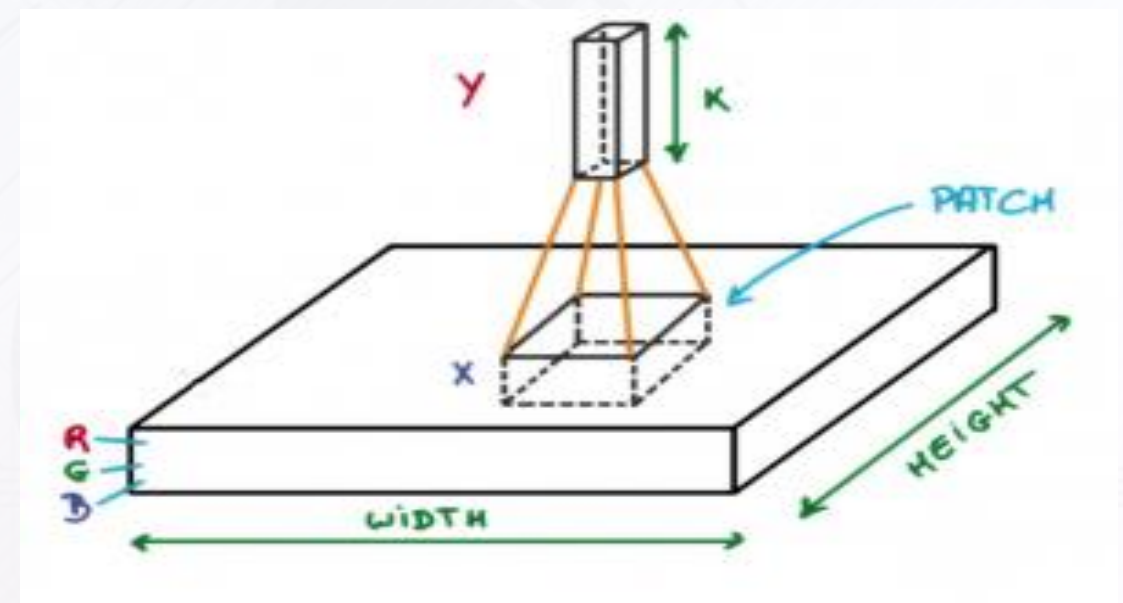


One **major problem** with computer vision problems is that the input data can get really big. Suppose an image is of the size 68 X 68 X 3. The input feature dimension then becomes 12,288. This will be even bigger if we have larger images (say, of size 720 X 720 X 3). Now, if we pass such a big input to a neural network, the number of parameters will swell up to a HUGE number

CNN

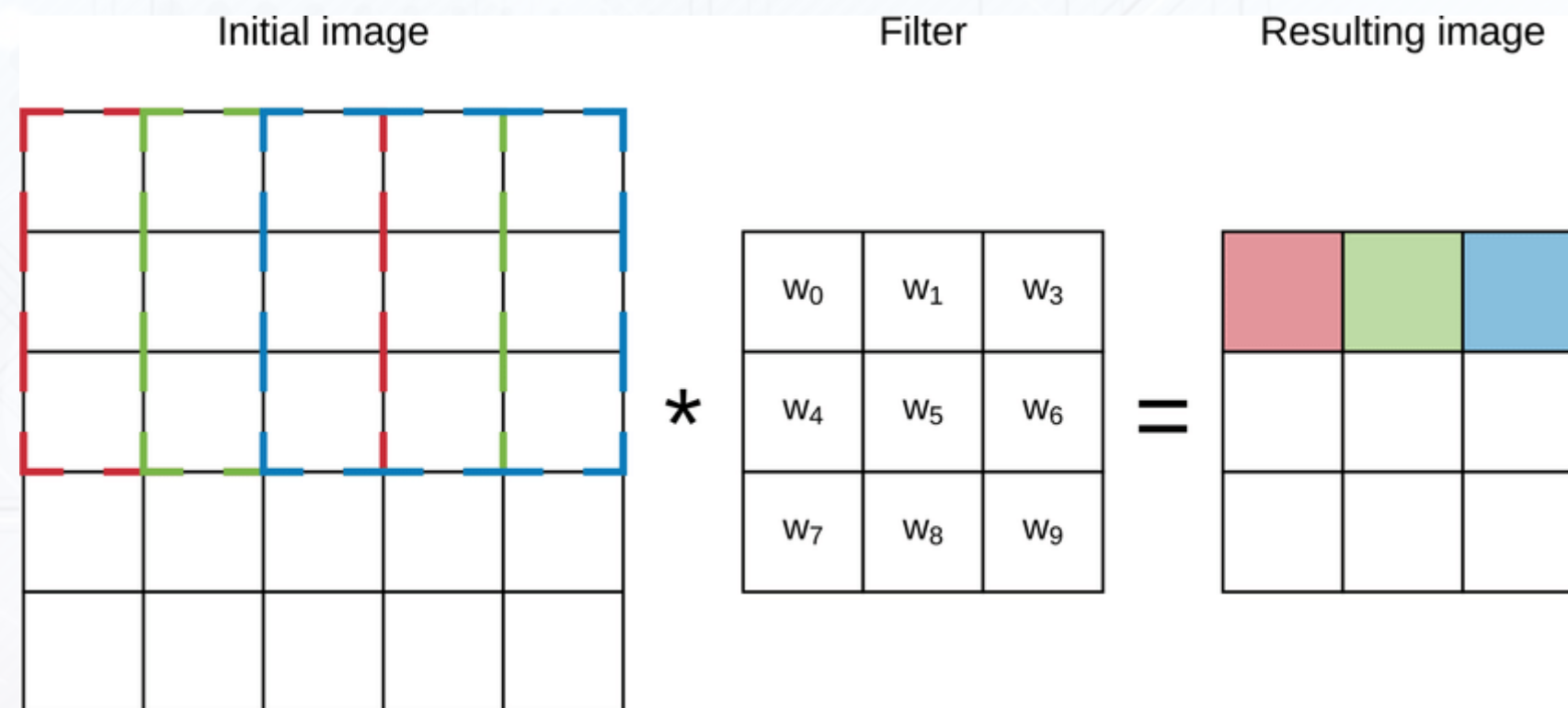
Now imagine taking a small patch of this image and running a **small neural network**, called a **filter or kernel** on it, with say, K outputs and representing them vertically.

Now slide that neural network across the whole image, as a result, we will get another image with different widths, heights, and depths. Instead of just R, G, and B channels now we have more channels but lesser width and height. This operation is called **Convolution**.



CNN

Kernel in CNN



CNN

1. Function of Kernels

Kernels slide over the input data (e.g., an image), performing element-wise multiplication followed by a summation of the results. This process effectively extracts specific features from the input, such as edges, corners, or textures, depending on the kernel's values. Each kernel is designed to detect a specific type of feature at various locations in the input.

2. Kernel Structure

Size: Kernels are typically small (e.g., 3×3 , 5×5 , or 7×7 matrices) compared to the size of the input data. The size of the kernel affects how much of the input data is considered at one time for any given feature extraction operation.

CNN

Depth: The depth of a kernel in a CNN corresponds to the depth of the input volume. For example, if the input data is a color image with three color channels (RGB), the kernel will also have three channels.

3. Learning Process

In CNNs, the values in the kernels are not predetermined but are learned during the training process. Through backpropagation and optimization algorithms like gradient descent, the CNN adjusts the values in the kernels to minimize the loss function of the network. This learning process allows the kernels to become better at extracting useful features that help the network achieve good performance on its task.

CNN

4. Feature Maps

The output of the convolution operation is called a feature map or activation map. This output demonstrates what features were detected in the input. For instance, one kernel might produce a feature map highlighting the vertical edges, while another might highlight horizontal edges.

5. Stacking Multiple Kernels

Multiple kernels are typically used at each layer of a CNN, allowing the network to extract various features at each layer. The outputs (feature maps) from these kernels can be stacked to form the input for the next layer, creating a hierarchy of features from simple to complex as you move deeper into the network.

CNN

More Edge Detection

The type of filter that we choose helps to detect the vertical or horizontal edges. We can use the following filters to detect different edges:

1	0	-1
1	0	-1
1	0	-1

Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

Some of the commonly used filters are:

1	0	-1
2	0	-2
1	0	-1

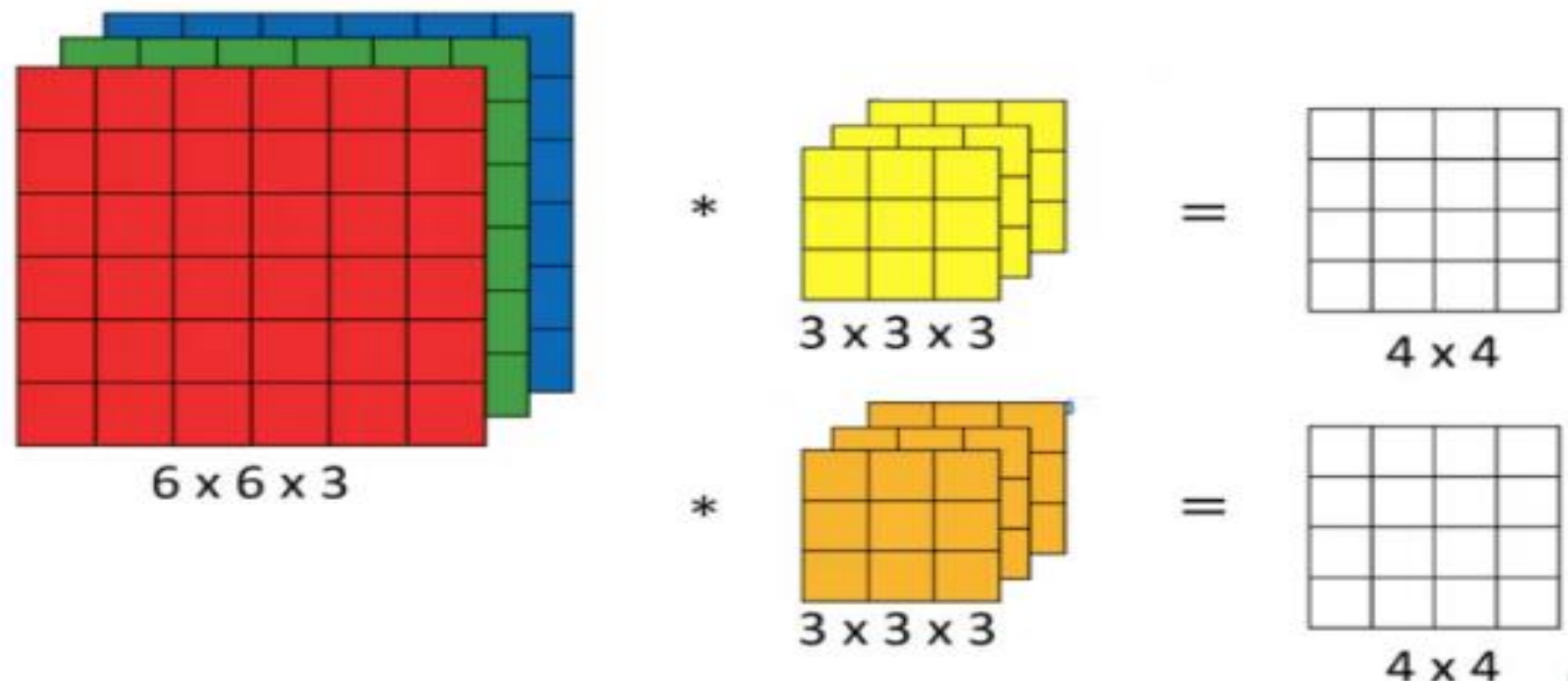
Sobel
filter

3	0	-3
10	0	-10
3	0	-3

Scharr
filter

CNN

Instead of using just a single filter, we can use multiple filters as well. How do we do that? Let's say the first filter will detect vertical edges and the second filter will detect horizontal edges from the image. If we use multiple filters, the output dimension will change. So, instead of having a 4 X 4 output as in the above example, we would have a 4 X 4 X 2 output (if we have used 2 filters):



CNN

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

=

6		

$$\begin{aligned}
 &7 \times 1 + 4 \times 1 + 3 \times 1 + \\
 &2 \times 0 + 5 \times 0 + 3 \times 0 + \\
 &3 \times -1 + 3 \times -1 + 2 \times -1 \\
 &= 6
 \end{aligned}$$

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

=

6	-9	-8
-3	-2	-3
-3	0	-2

How Kernels Operate in a Convolutional Neural Network?

1. Initial Placement

The kernel, which is a small matrix of weights, begins its process at the top-left corner of the input image. This initial placement ensures that every part of the image is systematically scanned by the kernel.

2. Dot Product Calculation

Element-wise Multiplication: As the kernel is placed over a specific part of the image, each element of the kernel is multiplied by the corresponding element of the input image it covers.

Summation: The results of these multiplications are then summed together to produce a single output value. This sum represents how much the section of the image matches the pattern defined by the kernel's weights.

How Kernels Operate in a Convolutional Neural Network?

. Recording the Output

Feature Map Formation: The single output value obtained from the dot product calculation is stored in a feature map. This value corresponds to the specific location on the feature map that mirrors the kernel's current position on the input image.

Response Interpretation: This output value (or activation) indicates the strength of the response between the image section and the kernel, highlighting features like edges, textures, or other patterns that the kernel is designed to detect.

How Kernels Operate in a Convolutional Neural Network?

4. Sliding the Kernel

Convolution: The kernel slides across the entire image to systematically apply the same operation to every possible position on the input. This sliding action is controlled by a parameter known as the stride, which dictates how many pixels the kernel moves each time (commonly one or two pixels).

Edge Coverage: When the kernel reaches the edges of the image, handling strategies such as padding (typically zero-padding) are employed to allow complete coverage of the image edges and corners. Padding involves adding extra pixels of zeros around the edge of the image so that the kernel can fit properly.

Padding

We have seen that convolving an input of 6×6 dimension with a 3×3 filter results in 4×4 output. We can generalize it and say that if the input is $n \times n$ and the filter size is $f \times f$, then the output size will be $(n-f+1) \times (n-f+1)$:

Input: $n \times n$

Filter size: $f \times f$

Output: $(n-f+1) \times (n-f+1)$

Padding

There are primarily two disadvantages here:

Every time we apply a convolutional operation, the size of the image shrinks

Pixels present in the corner of the image are used only a few number of times during convolution as compared to the central pixels. Hence, we do not focus too much on the corners since that can lead to information loss

Padding

There are primarily two disadvantages here:

Every time we apply a convolutional operation, the size of the image shrinks

Pixels present in the corner of the image are used only a few number of times during convolution as compared to the central pixels. Hence, we do not focus too much on the corners since that can lead to information loss

INPUT IMAGE

18	54	51	239	244	188
55	121	75	78	95	88
35	24	204	113	109	221
3	154	104	235	25	130
15	253	225	159	78	233
68	85	180	214	245	0

0	0	0	0	0	0	0	0
0	18	54	51	239	244	188	0
0	55	121	75	78	95	88	0
0	35	24	204	113	109	221	0
0	3	154	104	235	25	130	0
0	15	253	225	159	78	233	0
0	68	85	180	214	245	0	0
0	0	0	0	0	0	0	0

Stride

Suppose we choose a stride of 2. So, while convoluting through the image, we will take two steps – both in the horizontal and vertical directions separately. The dimensions for stride s will be:

Input: $n \times n$

Padding: p

Stride: s

Filter size: $f \times f$

Output: $[(n+2p-f)/s+1] \times [(n+2p-f)/s+1]$

Stride helps to reduce the size of the image, a particularly useful feature.

Stride

Stride= 2

As you can see the size of image keeps on reducing as we increase the stride value.

Padding the input image with zeros across it solves this problem for us. We can also add more than one layer of zeros around the image in case of higher stride values.

INPUT IMAGE				
18	54	51	239	244
55	121	75	78	95
35	24	204	113	109
3	154	104	235	25
15	253	225	159	78

WEIGHT		
1	0	1
0	1	0
1	0	1

429

INPUT IMAGE				
18	54	51	239	244
55	121	75	78	95
35	24	204	113	109
3	154	104	235	25
15	253	225	159	78

WEIGHT		
1	0	1
0	1	0
1	0	1

429 686

INPUT IMAGE				
18	54	51	239	244
55	121	75	78	95
35	24	204	113	109
3	154	104	235	25
15	253	225	159	78

WEIGHT		
1	0	1
0	1	0
1	0	1

429 686
633

INPUT IMAGE				
18	54	51	239	244
55	121	75	78	95
35	24	204	113	109
3	154	104	235	25
15	253	225	159	78

WEIGHT		
1	0	1
0	1	0
1	0	1

429 686
633 412

Why do we use striding?

Adjusting the stride can affect the output size and computational efficiency of the network, influencing feature extraction and spatial dimensions of the data.

1. In some projects you don't need to extract all features from input datasets. You just need to extract high level features. In those cases, we need to use striding, because it specifies the jump of the kernel, the more striding will be applied, the more low level feature will be ignored.

2. Sometimes we have limited computing resources with comparison of the size of the datasets. The more striding will be applied, the more high-level feature it will extract, the less computing resource will be needed. Also, the time will decrease.